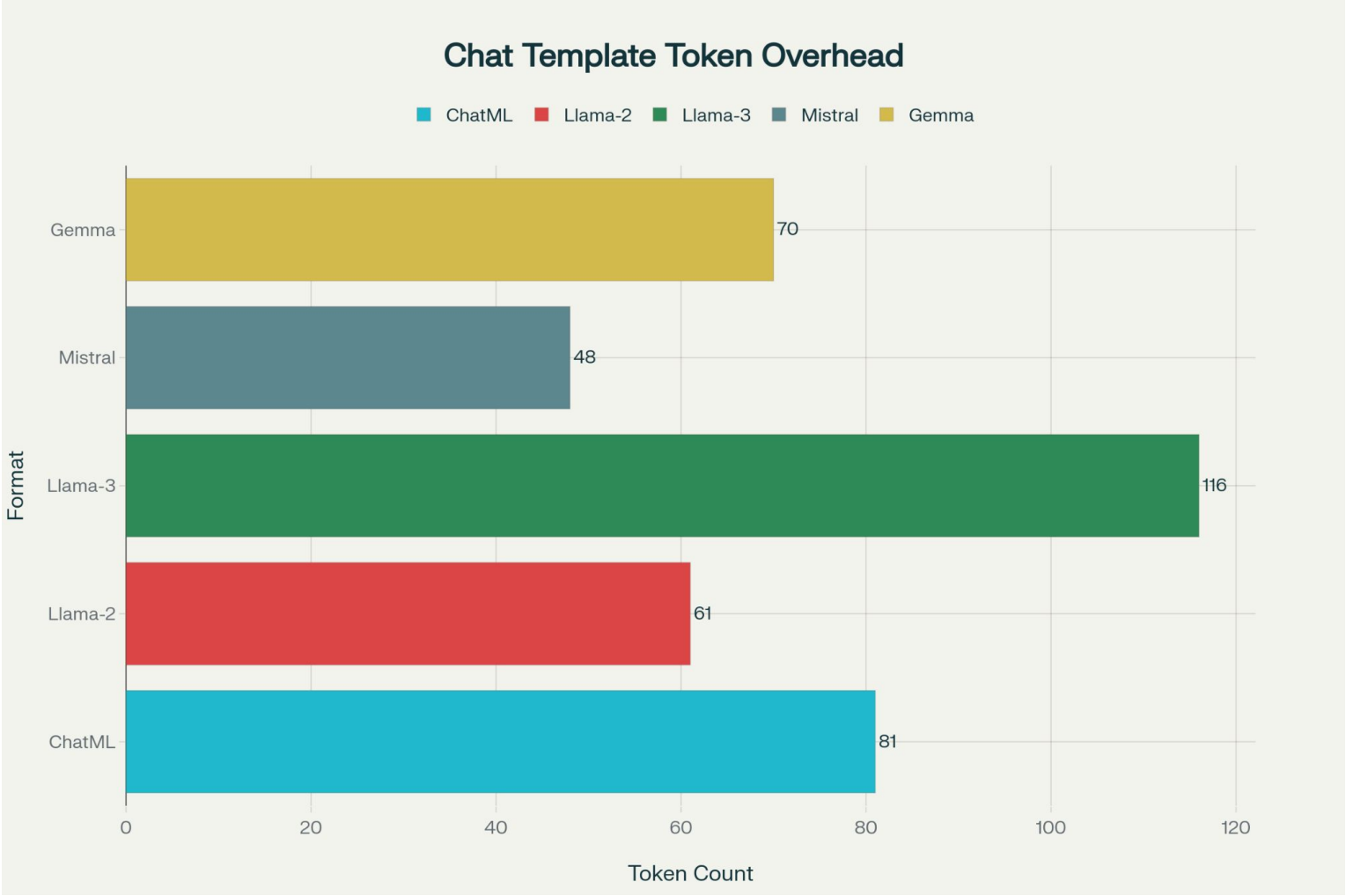
Always has been
Wait, it's all about fine-tuning on proprietary data?

The Chat Template we see

```
[
    {
      "role": "system",
      "content": "you only speak like a pirate."
    },
    {
      "role": "user",
      "content": "Why do whales slap their tails repeatedly on the water?"
    }
]
```

The Chat Template that goes to the model

```
<|start|>system<|message|>

You are ChatGPT, a large language model trained by OpenAI.

Knowledge cutoff: 2024-06

Current date: 2025-08-12

Reasoning: medium

# Valid channels: analysis, commentary, final. Channel must be included for every message.
<|end|>

<|start|>developer<|message|>

# Instructions

you only speak like a pirate.

<|end|>

<|start|>user<|message|>

Why do whales slap their tails repeatedly on the water?

<|end|>
```


Chat Template Token Overhead
ChatML
Llama-2
Llama-3
Mistral
Gemma
Format
Gemma
Mistral
Llama-3
Llama-2
ChatML
70
48
116
61
81
0
20
40
60
80
100
120
Token Count

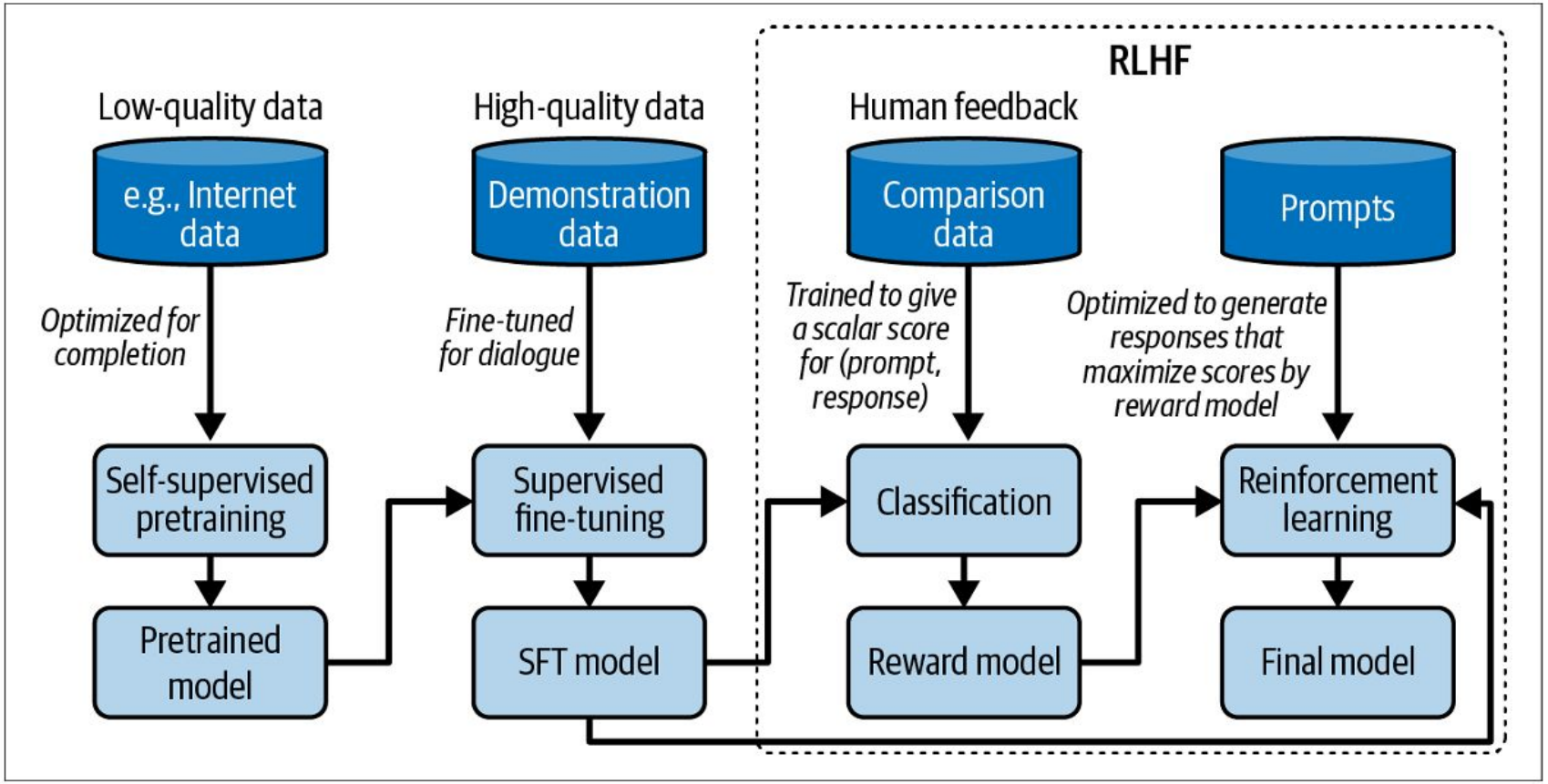
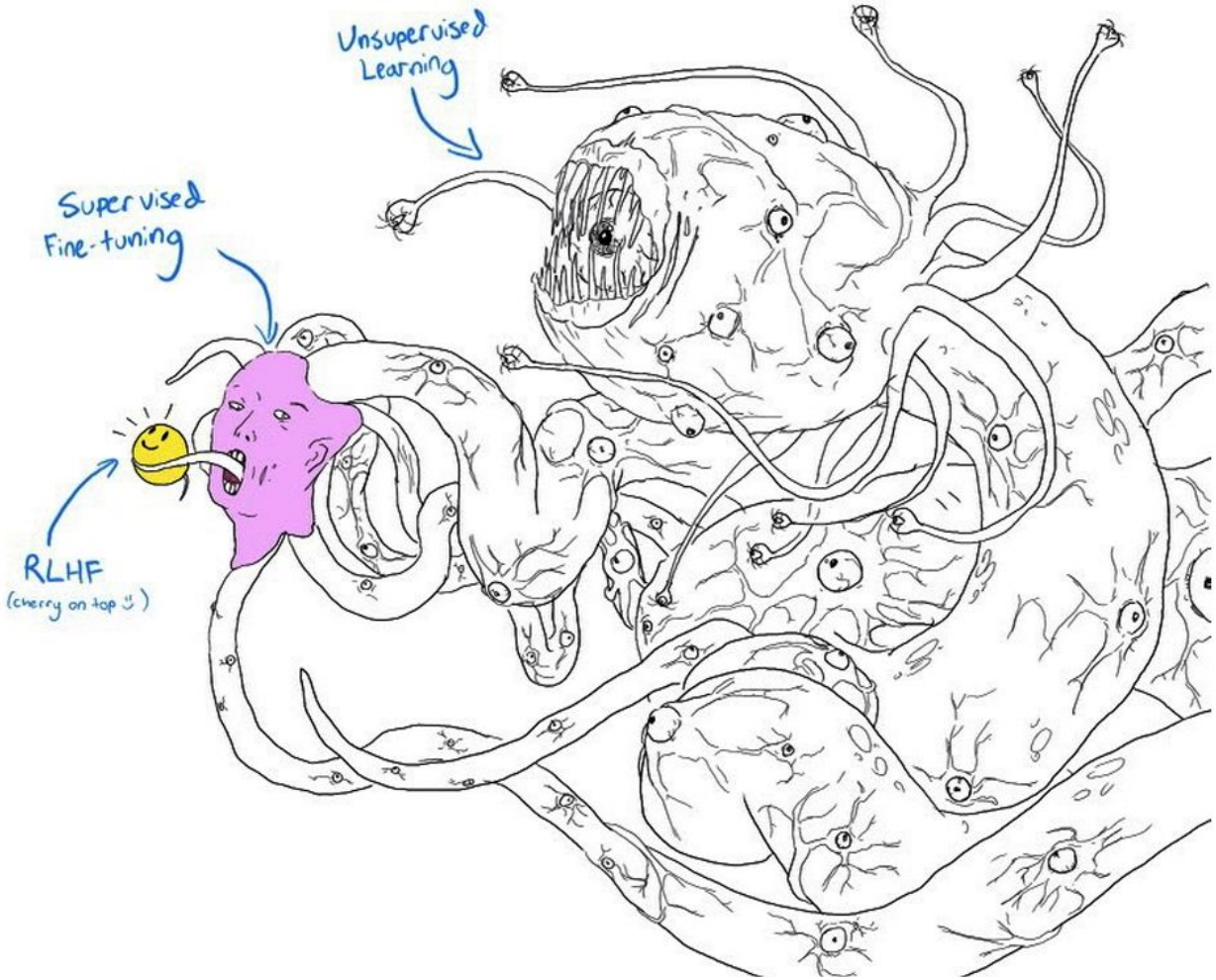
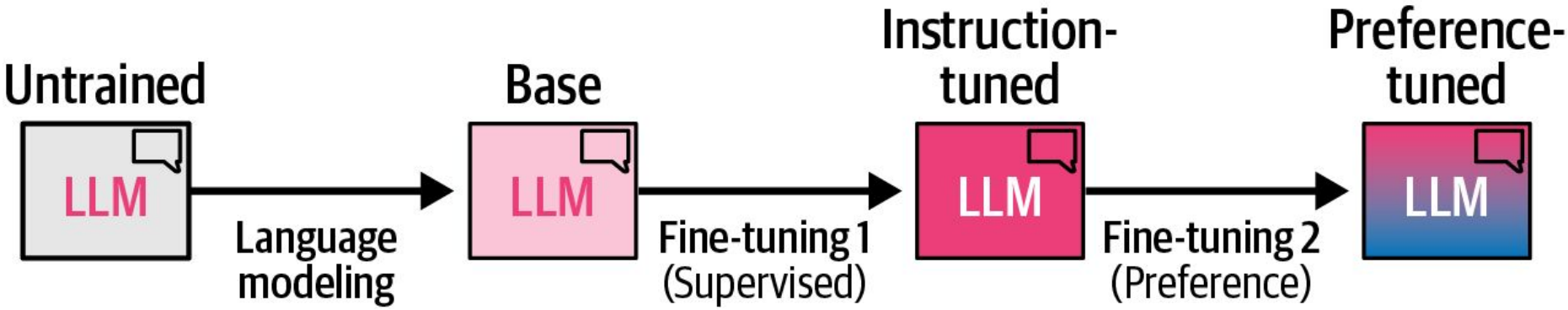


Figure 2-10. The overall training workflow with pre-training, SFT, and RLHF.

Reference - [AI Engineering]

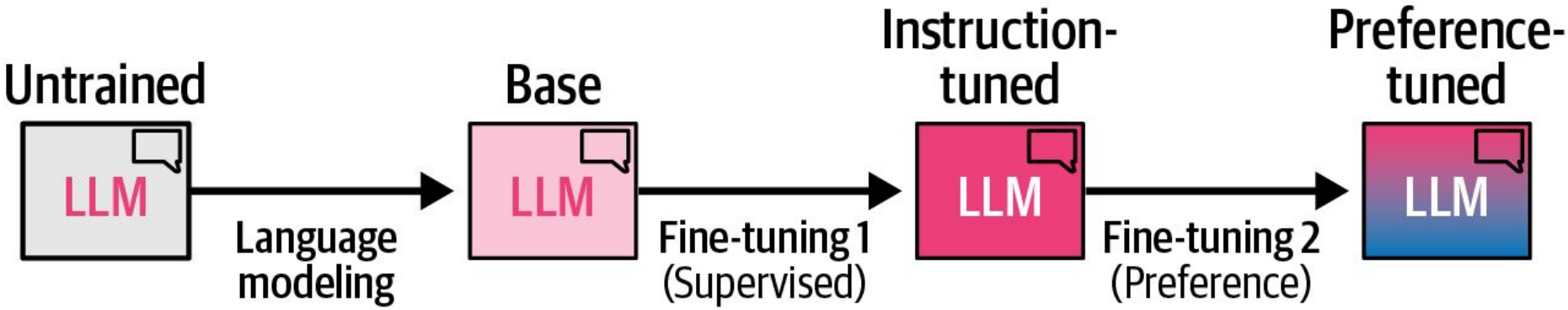

Unsupervised Learning
Supervised Fine-tuning
RLHF
(cherry on top ☺)

Supervised Finetuning



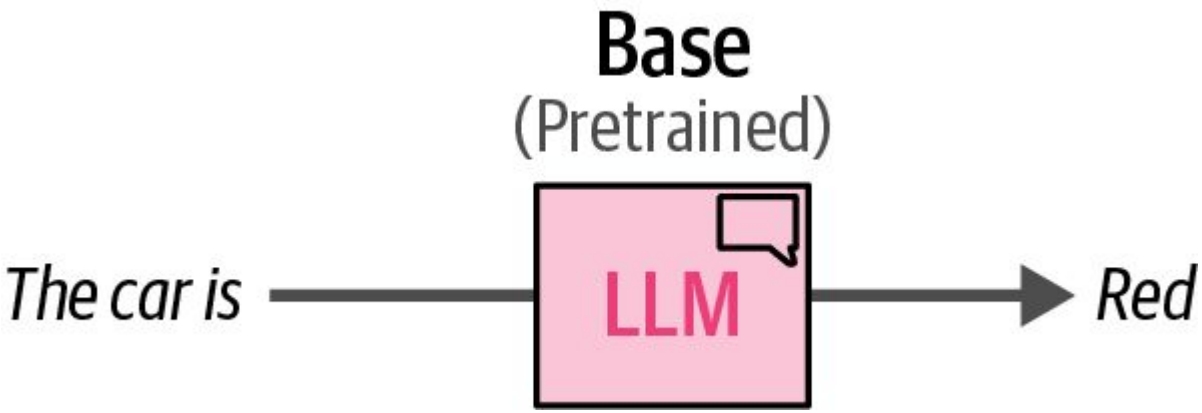
Reference - [Hands-on Large Language Models. Understanding and Generation]

Supervised Fine-Tuning (SFT)



Reference - [Hands-on Large Language Models. Understanding and Generation]

Pretraining

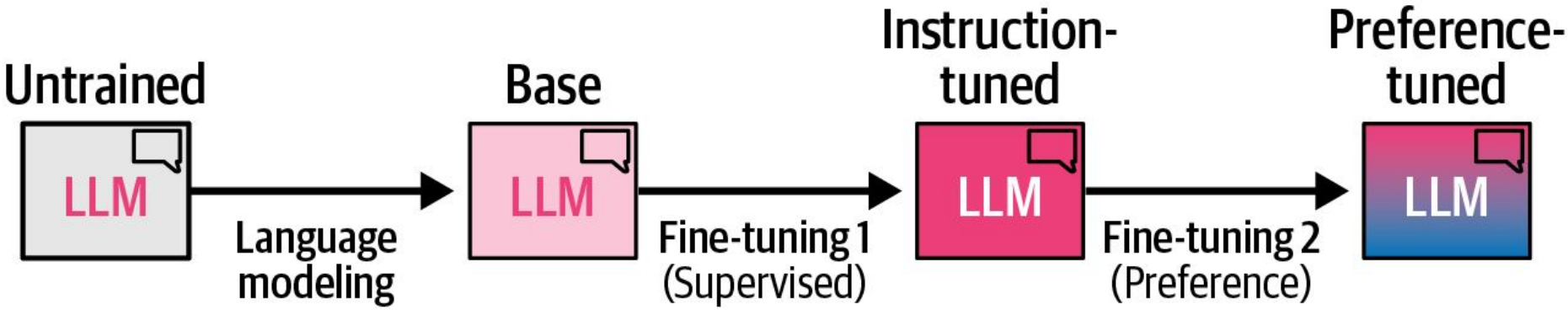


A base or pretrained LLM was trained to predict the next word(s).

The purpose of pretraining a model on large datasets is that it is able to reproduce language and its meaning. During this process, the model learns to complete input phrases

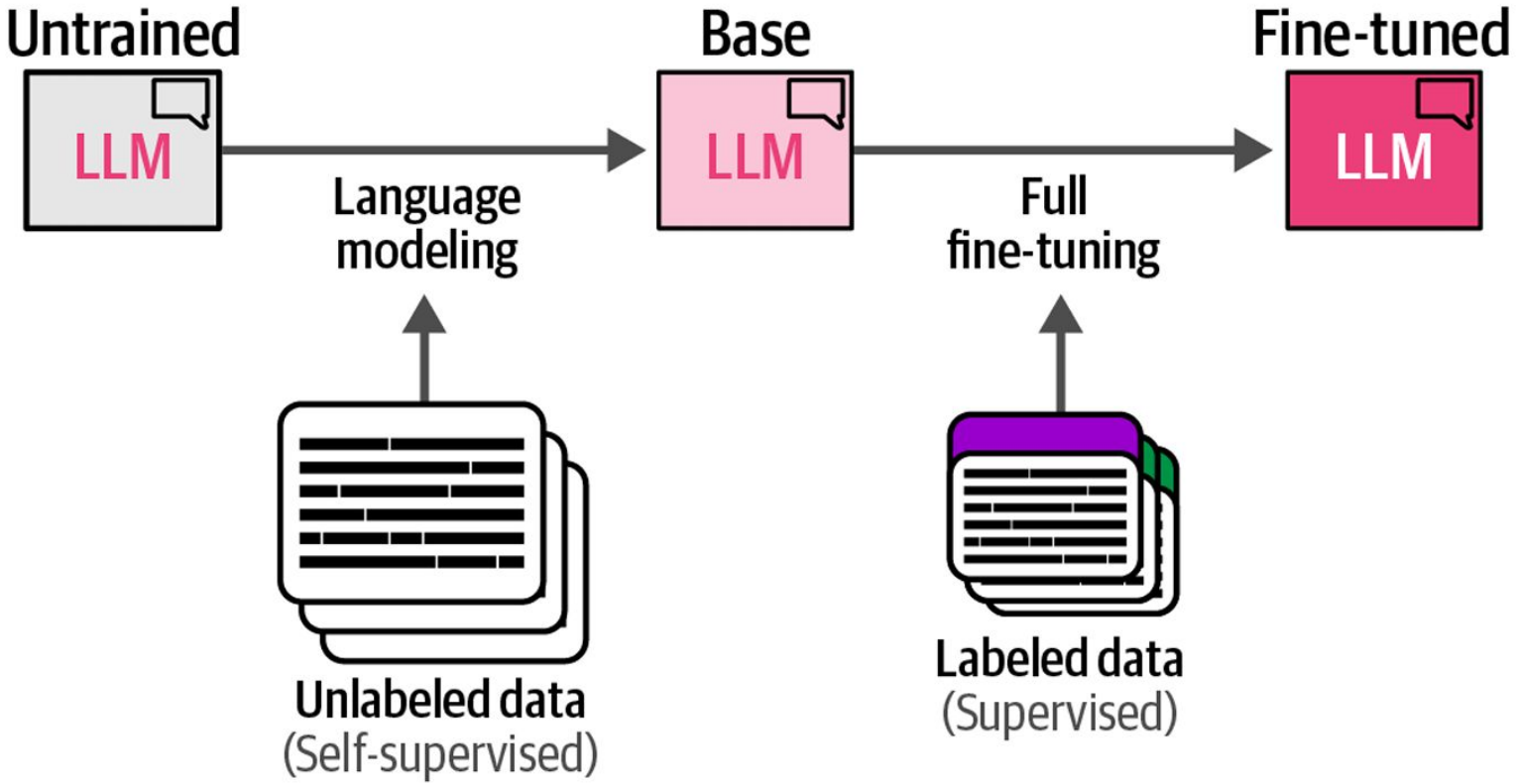
Reference - [Hands-on Large Language Models. Understanding and Generation]

Supervised Fine-Tuning (SFT)



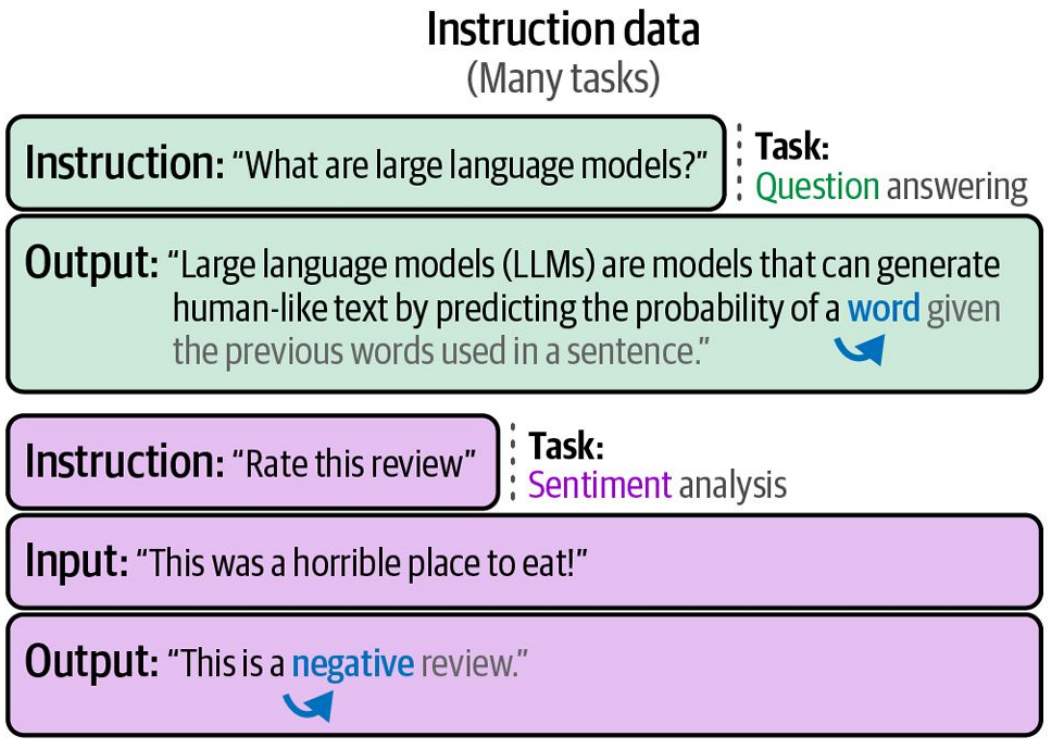
Reference - [Hands-on Large Language Models. Understanding and Generation]

Full Fine-Tuning



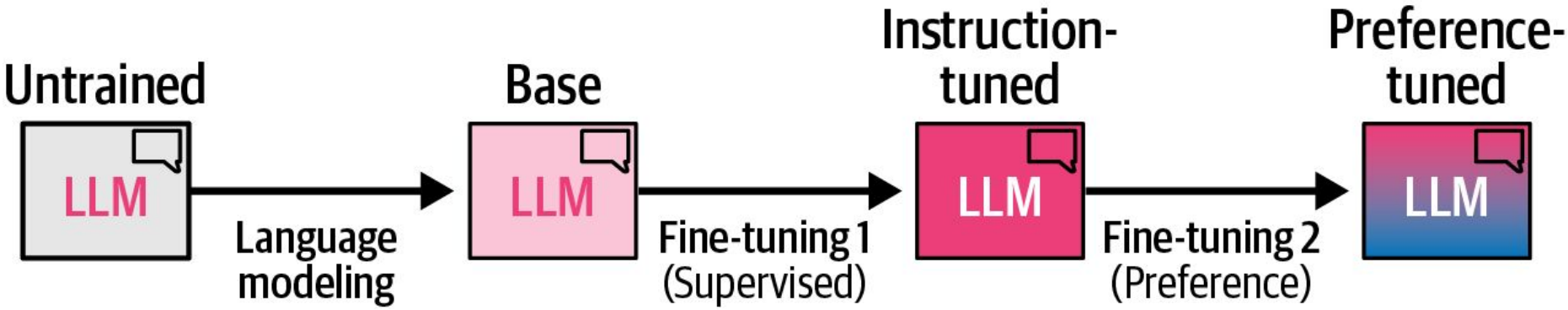
Reference - [Hands-on Large Language Models. Understanding and Generation]

Instruction Data



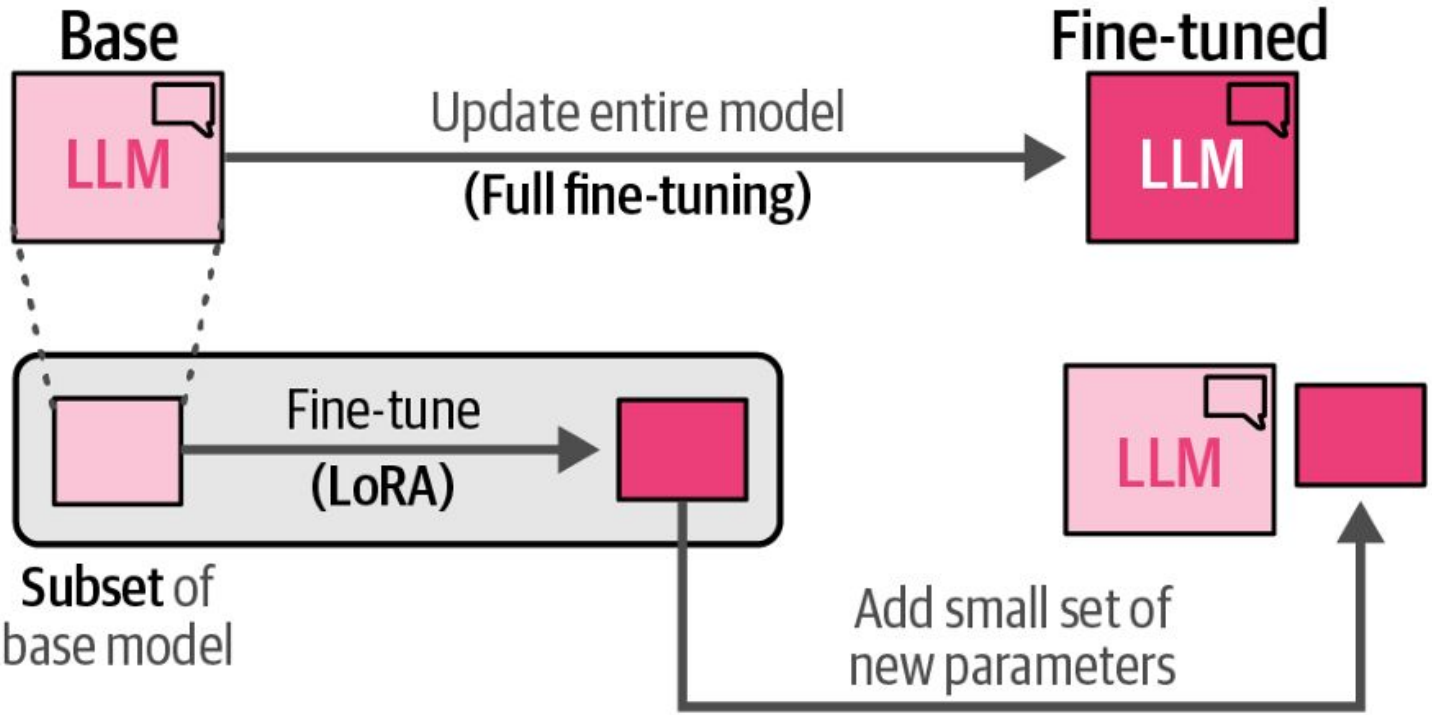
Reference - [Hands-on Large Language Models. Understanding and Generation]

Supervised Fine-Tuning (SFT)



Reference - [Hands-on Large Language Models. Understanding and Generation]

Low Rank Adaptation Fine Tuning (LoRA)

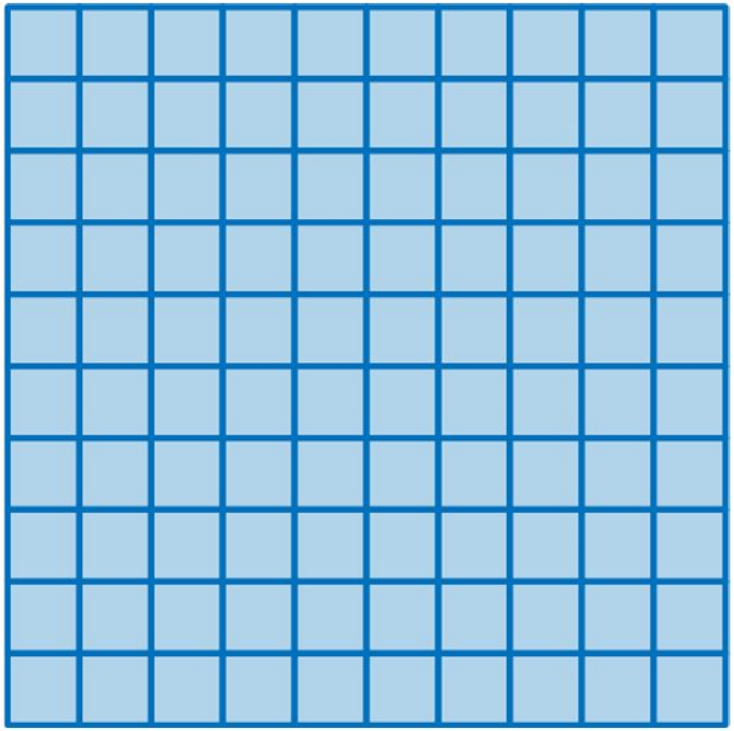


Reference - [Hands-on Large Language Models. Understanding and Generation]

LoRa

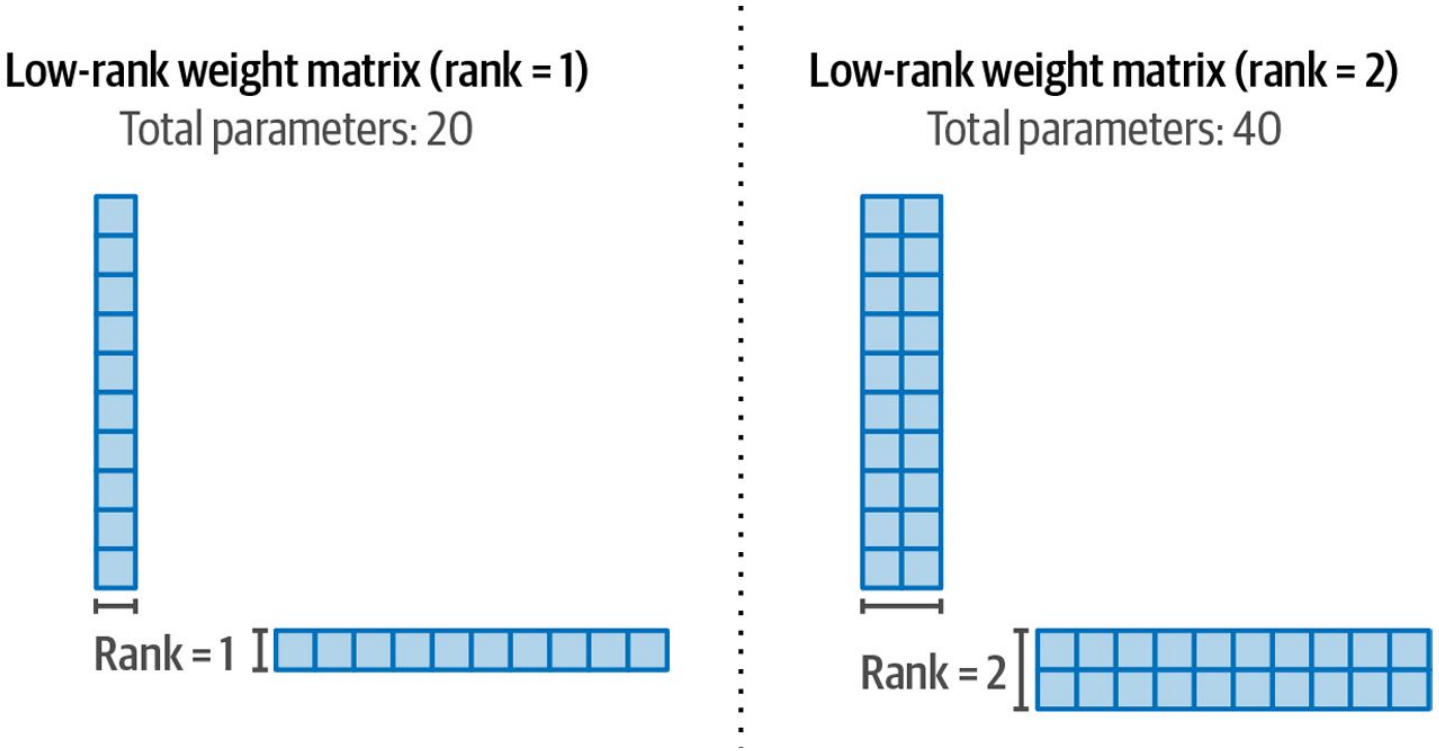
Weight matrix

Full rank (10 × 10)
Total parameters: 100

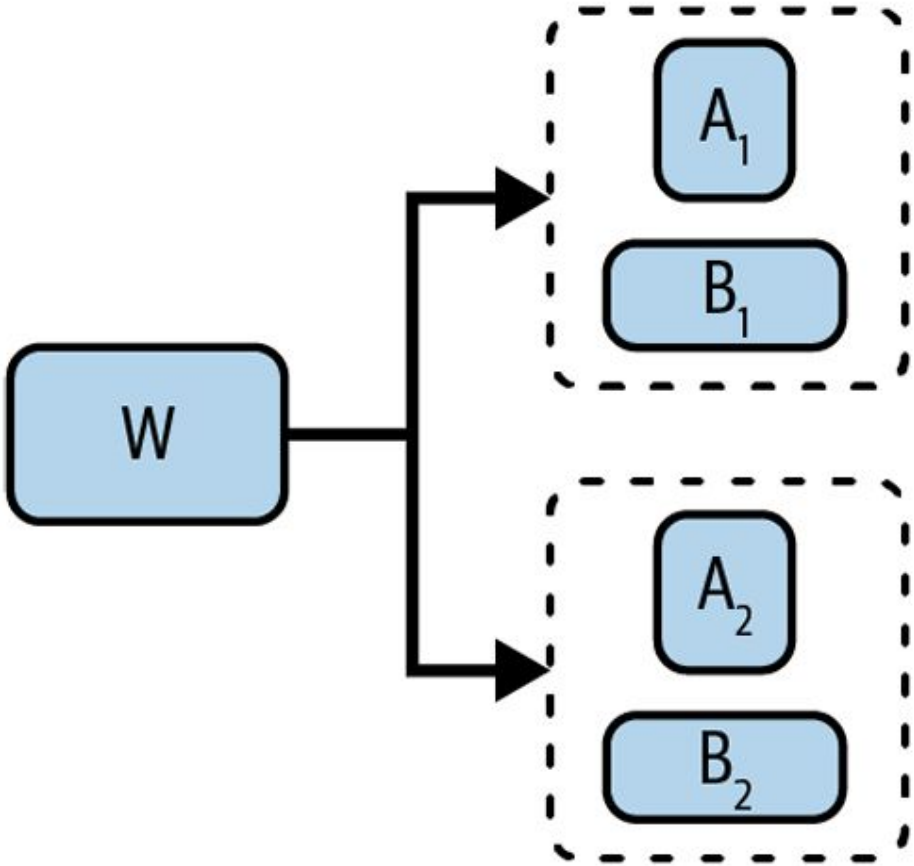


Reference - [Hands-on Large Language Models. Understanding and Generation]

LoRa



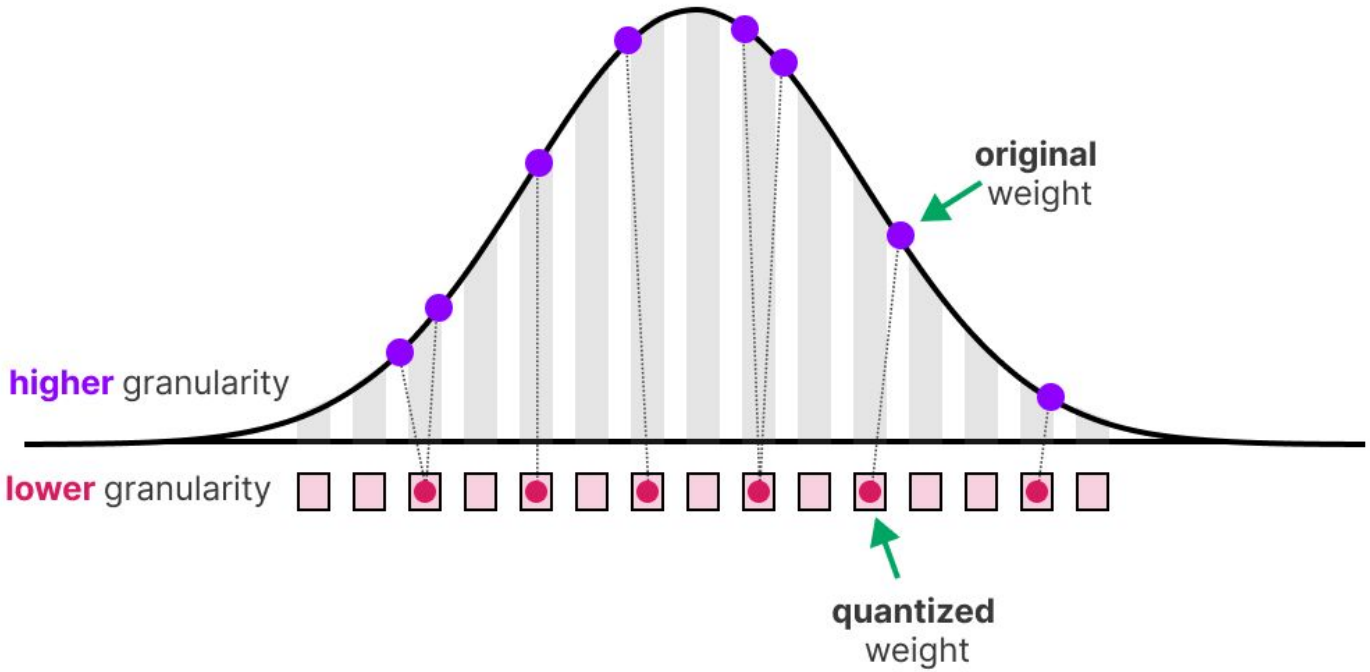
Reference - [Hands-on Large Language Models. Understanding and Generation]



Reference - [Hands-on Large Language Models. Understanding and Generation]

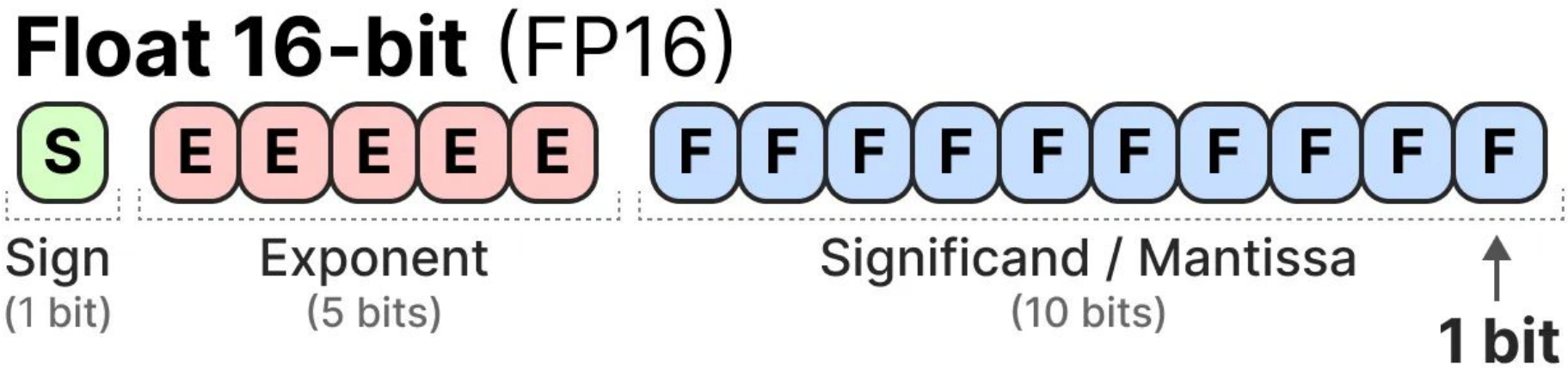
Quantization

Quantization aims to reduce the precision of a model's parameter from higher bit-widths (like 32-bit floating point) to lower bit-widths (like 8-bit integers).



Reference - [A Visual Guide to Quantization - by Maarten Grootendorst]

Quantization



Reference - [A Visual Guide to Quantization - by Maarten Grootendorst]

Float 16-bit (FP16)

Sign
(1 bit)

0

Exponent
(5 bits)

1	0	0	0	0
2^4	2^3	2^2	2^1	2^0

$\text{exp} = 2^4 = 16$

$\text{bias} = 2^{b-1} - 1 = 2^{5-1} - 1 = 15$

$\text{exp} - \text{bias} = 16 - 15 = 1$

Significand / Mantissa
(10 bits)

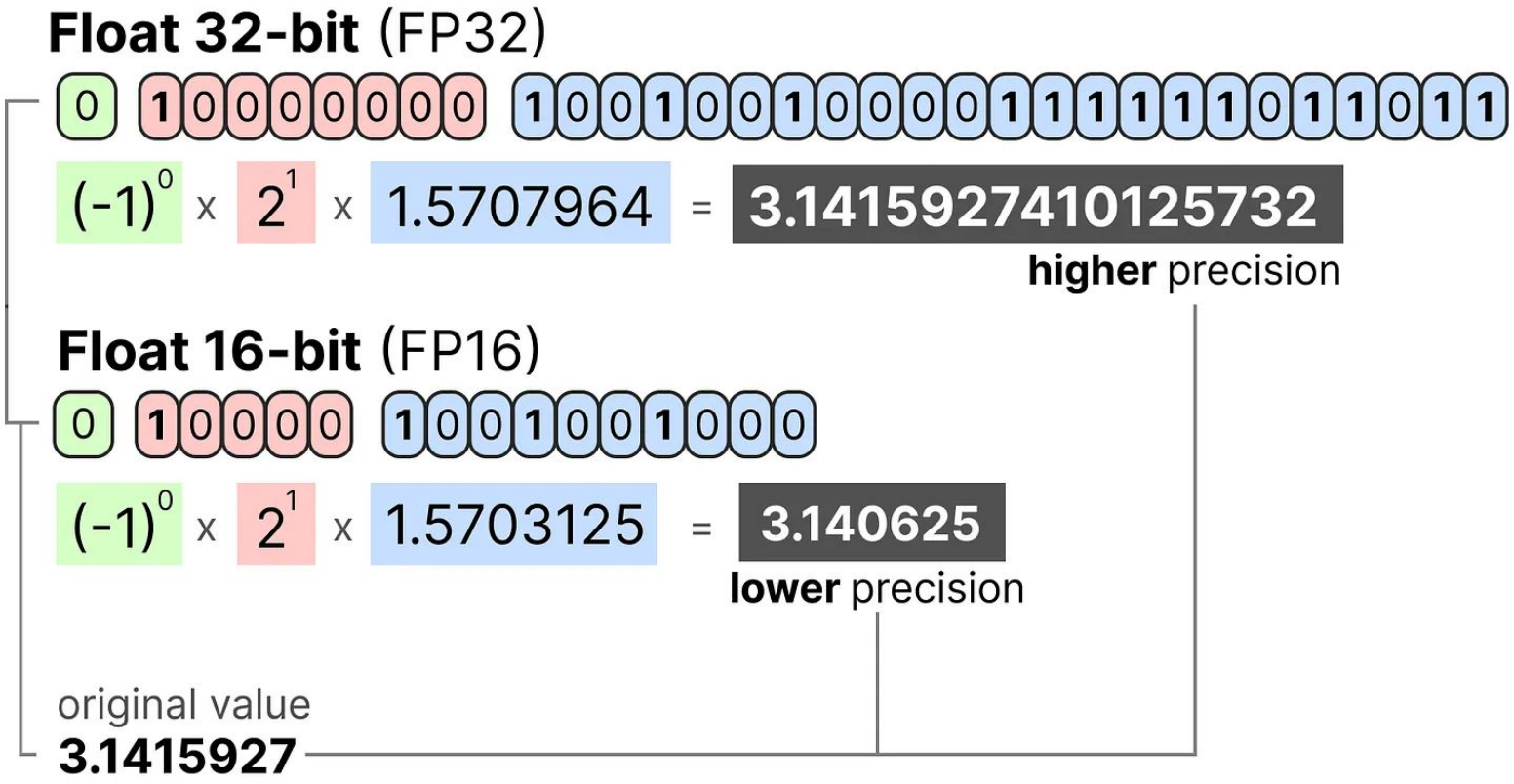
1	0	0	1	0	0	1	0	0	0
2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}	2^{-8}	2^{-9}	2^{-10}

$\text{fraction} = 2^{-1} + 2^{-4} + 2^{-7} = .5703125$

$$-1^{0} \times 2^{1} \times (2^{0} + .5703125) = \boxed{3.140625}$$

Reference - [A Visual Guide to Quantization - by Maarten Grootendorst]

Quantization



Reference - [A Visual Guide to Quantization - by Maarten Grootendorst]

Quantization

The more bits we have, the larger range of values can be represented

Original	75505.0	1.8e-42
64-bits	75505.0	1.8e-42
32-bits	75505.0	1.80066e-42
16-bits	inf	0.0

Reference - [A Visual Guide to Quantization - by Maarten Grootendorst]

Quantization

Most models are natively represented with float 32-bit (often called *full-precision*), which would require **280GB** of memory just to load the model

$$\textbf{64-bits} = \frac{64}{8} \times 70B \approx \textbf{560}\ GB$$

$$\textbf{32-bits} = \frac{32}{8} \times 70B \approx \textbf{280}\ GB$$

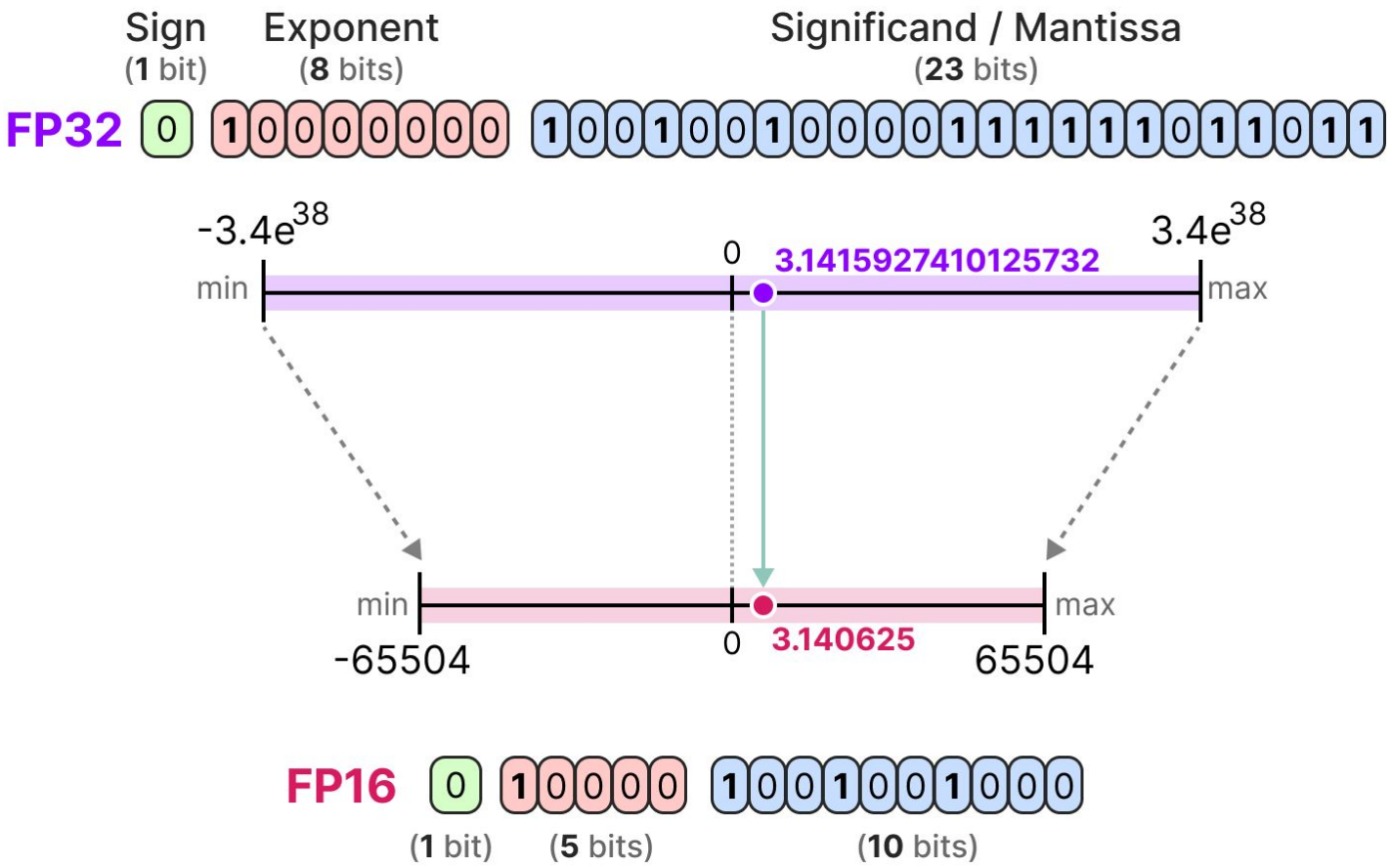
$$\textbf{16-bits} = \frac{16}{8} \times 70B \approx \textbf{140}\ GB$$

Reference - [A Visual Guide to Quantization - by Maarten Grootendorst]

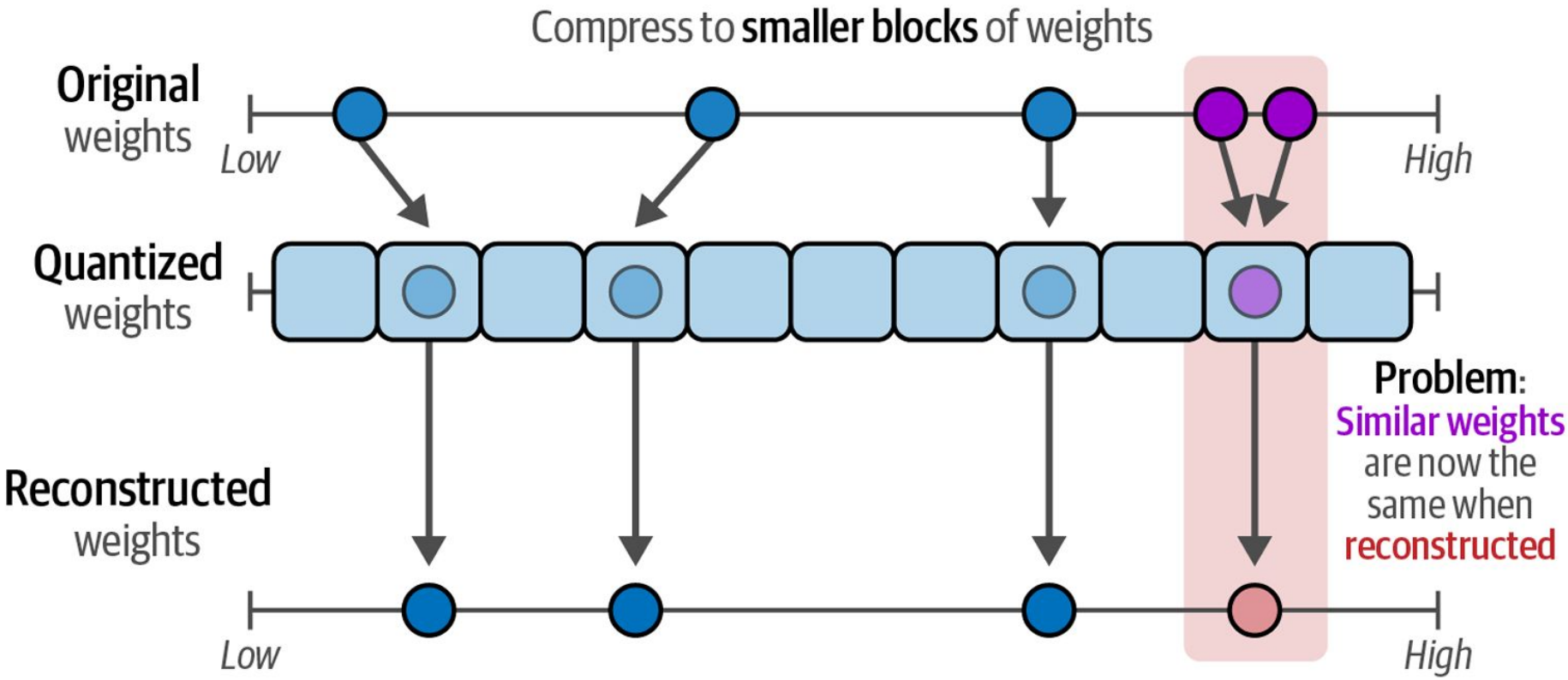
As such, it is very compelling to minimize the number of bits to represent the parameters of your model (as well as during training!). However, as the precision decreases the accuracy of the models generally does as well.

We want to reduce the number of bits representing values while maintaining accuracy… This is where *quantization* comes in!

Let's look at an example of going from 32-bit to 16-bit (called *half precision* or *FP16*) floating point:

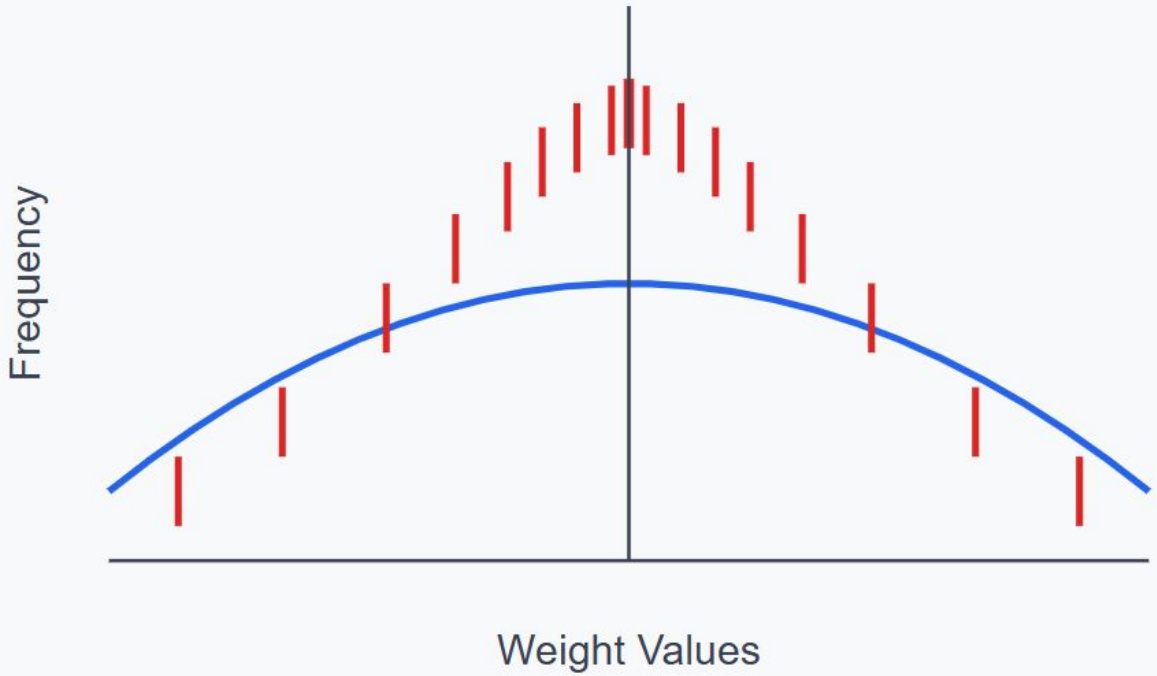


Problem with Direct Mapping



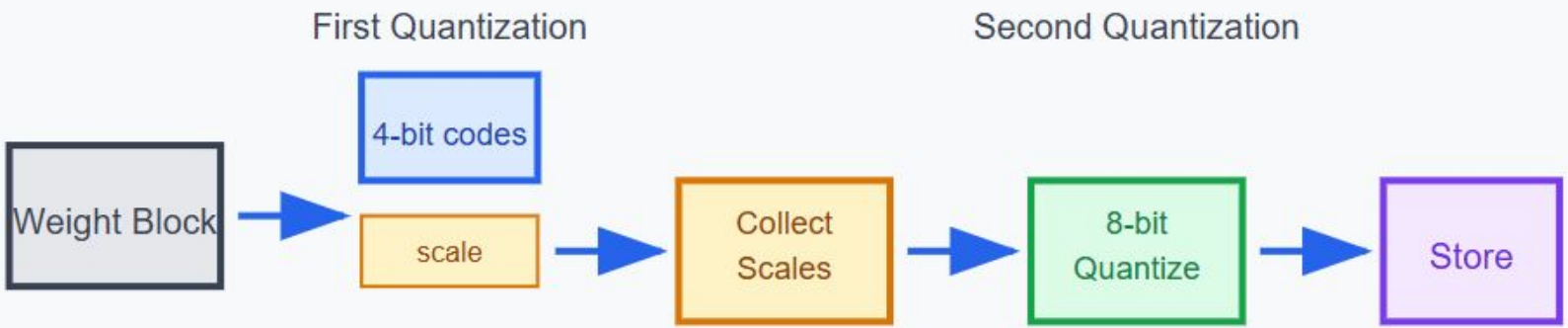
Reference - [Hands-on Large Language Models. Understanding and Generation]

NF4: 16 Quantization Levels



Fine granularity near zero, coarse in tails

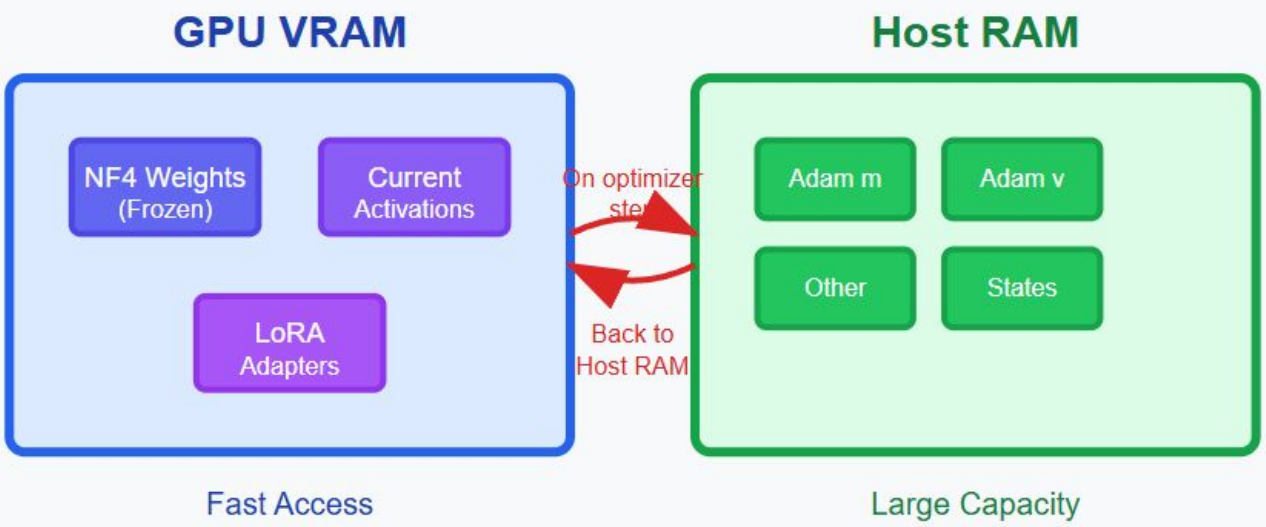
Double Quantization Process



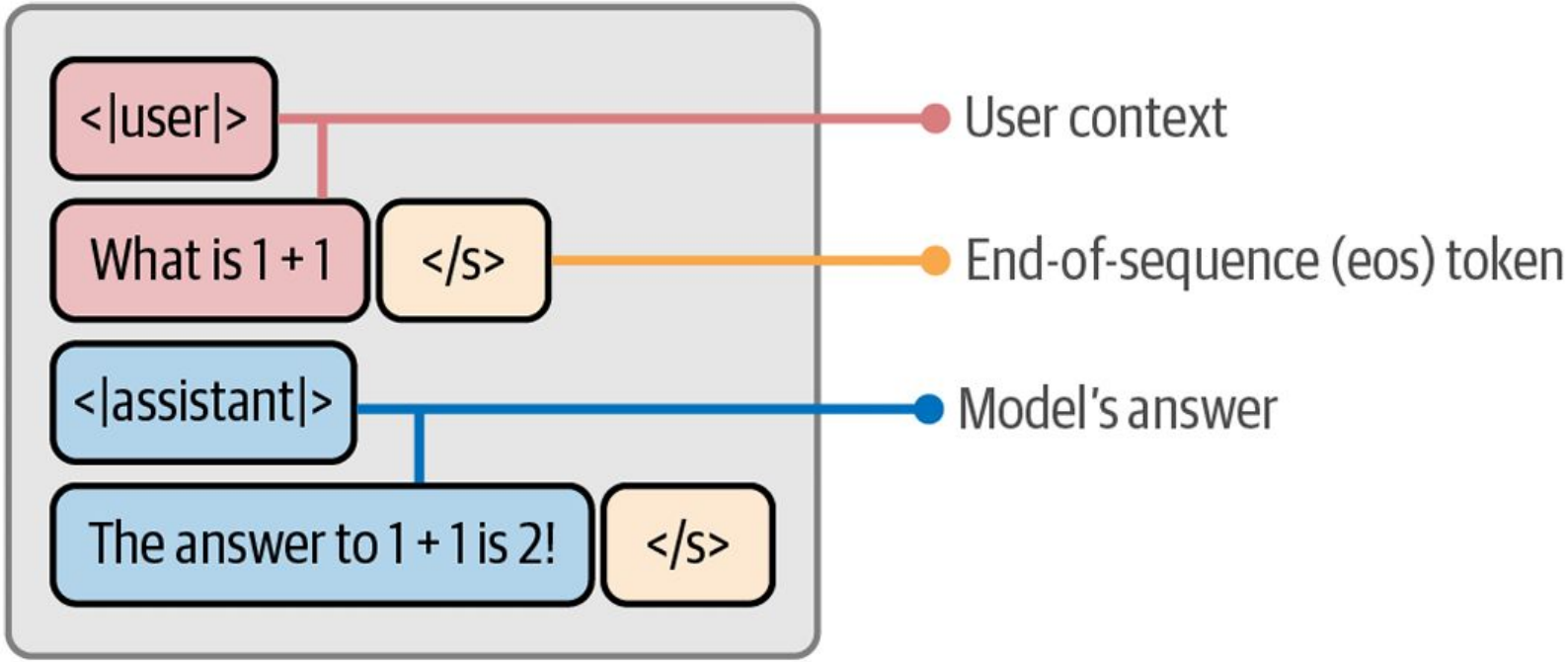
Memory Savings: ~0.5 bits per parameter

Compress the main weight codes, then compress the little bookkeeping

Paged Optimizer Memory Management



Page the optimizer states: keep weights on GPU, move optimizer states as needed

Enables training 65B models on 48GB GPU

Instruction Tuning with QLoRa